

此前的搜索树结构和词典结构，都支持覆盖数据全集的访问和操作。也就是说，其中存储的每一数据对象都可作为查找和访问目标。为此，搜索树结构需要在所有元素之间定义并维护一个显式的全序（**full order**）关系；而词典结构中的数据对象之间，尽管不必支持比较大小，但在散列表之类的具体实现中，都从内部强制地在对象的数值与其对应的秩之间，建立起某种关联（尽管实际上这种关联通常越“随机”越好），从而隐式地定义了一个全序次序。

就对外接口的功能而言，本章将要介绍的优先级队列，较之此前的数据结构反而有所削弱。具体地，这类结构将操作对象限定于当前的全局极值者。比如，在全体北京市民中，查找年龄最长者；或者在所有鸟类中，查找种群规模最小者，等等。这种根据数据对象之间相对优先级对其进行访问的方式，与此前的访问方式有着本质区别，称作循优先级访问（**call-by-priority**）。

当然，“全局极值”本身就隐含了“所有元素可相互比较”这一性质。然而，优先级队列并不会也不必忠实地动态维护这个全序，却转而维护一个偏序（**partial order**）关系。其高明之处在于，如此不仅足以高效地支持仅针对极值对象的接口操作，更可有效地控制整体计算成本。正如我们将要看到的，对于常规的查找、插入或删除等操作，优先级队列的效率并不低于此前的结构；而对于数据集的批量构建及相互合并等操作，其性能却更胜一筹。作为不失高效率的轻量级数据结构，优先级队列在许多领域都是扮演着不可替代的角色。

§ 10.1 优先级队列ADT

10.1.1 优先级与优先级队列

除了作为存放数据的容器，数据结构还应能够按某种约定的次序动态地组织数据，以支持高效的查找和修改操作。比如4.5节的队列结构，可用以描述和处理日常生活中的很多问题：在银行排队等候接受服务的客户，提交给网络打印机的打印任务等，均属此列。在这类问题中，无论客户还是打印任务，接受服务或处理的次序完全取决于其出现的时刻——先到的客户优先接受服务，先提交的打印任务优先执行——此即所谓“先进先出”原则。

然而在更多实际应用环境中，这一简单公平的原则并不能保证整体效率必然达到最高。试想，若干病人正在某所医院的门诊处排队等候接受治疗，忽然送来一位骨折的病人。要是固守“先进先出”的原则，那么他只能咬牙坚持到目前已经到达的每位病人都已接受过治疗之后。显然，那样的话该病人将承受更长时间的痛苦，甚至贻误治疗的最佳时机。因此，医院在此时都会灵活变通，优先治疗这位骨折的病人。同理，若此时又送来一位心脏病突发的患者，那么医生肯定也会暂时把骨折病人放在一边（如果没有更多医生的话），转而优先抢救心脏病病人。

由此可见，在决定病人接受治疗次序时，除了他们到达医院的先后次序，更应考虑到病情的轻重缓急，优先治疗病情最为危重的病人。在数据结构与算法设计中，类似的例子也屡见不鲜。在3.5.3节的选择排序算法中，每一步迭代都要调用**selectMax()**，从未排序区间选出最大者。在5.5.3节的Huffman编码算法中，每一步迭代都要调用**minHChar()**，从当前的森林中选出权重

最小的超字符。在基于空间扫描策略的各种算法中，每一步迭代都要根据到当前扫描线的距离，取出并处理最近的下一个事件点。

从数据结构的角度看，无论是待排序节点的数值、超字符的权重，还是事件的发生时间，数据项的某种属性只要可以相互比较大小，则这种大小关系即可称作优先级（priority）。而按照事先约定的优先级，可以始终高效查找并访问优先级最高数据项的数据结构，也统称作优先级队列（priority queue）。

10.1.2 关键码、比较器与偏序关系

仿照词典结构，我们也将优先级队列中的数据项称作词条（entry）；而与特定优先级相对应的数据属性，也称作关键码（key）。不同应用中的关键码，特点不尽相同：有时限定词条的关键码须互异，有时则允许词条的关键码雷同；有些词条的关键码一成不变，有些则可动态修改；有的关键码只是一个数字、一个字符或一个字符串，而复杂的关键码则可能由多个基本类型组合而成；多数关键码都取作词条内部的某一成员变量，而有的关键码则并非词条的天然属性。

无论具体形式如何，作为确定词条优先级的依据，关键码之间必须可以比较大小——注意，这与词典结构完全不同，后者仅要求关键码支持判等操作。因此对于优先级队列，必须以比较器的形式兑现对应的优先级关系。出于简化的考虑，与此前各章一样，本章依然假定关键码或者可直接比较，或者已重载了对应的操作符。

需特别留意的另一点是，尽管定义了明确的比较器即意味着在任何一组词条之间定义了一个全序关系，但正如2.7节所指出的，严格地维护这样一个全序关系必将代价不菲。实际上，优先级队列作为一类独特数据结构的意义恰恰在于，通过转而维护词条集的一个偏序关系。如此，不仅依然可以支持对最高优先级词条的动态访问，而且可将相应的计算成本控制在足以令人满意的范围之内。

10.1.3 操作接口

优先级队列接口的定义说明如表10.1所示。

表10.1 优先级队列ADT支持的操作接口

操 作 接 口	功 能 描 述
size()	报告优先级队列的规模，即其中词条的总数
insert()	将指定词条插入优先级队列
getMax()	返回优先级最大的词条（若优先级队列非空）
delMax()	删除优先级最大的词条（若优先级队列非空）

需要说明的是，本章允许在同一优先级队列中出现关键码雷同的多个词条，故insert()操作必然成功，因此该接口自然不必返回操作成功标志。

10.1.4 操作实例：选择排序

即便仍不清楚其具体实现，我们也已经可以按照以上ADT接口，基于优先级队列描述和实现各种算法。比如，实现和改进3.5.3节所介绍的选择排序算法。

具体的构思如下：将待排序的词条组织为一个优先级队列，然后反复调用delMax()接口，即可按关键码由大而小的次序逐一输出所有词条，从而得到全体词条的排序序列。

例如，针对某7个整数的这一排序过程，如表10.2所示。

表10.2 优先级队列操作实例：选择排序（当前的最大元素以方框示意）

操 作	优 先 级 队 列	输 出
initialization	{ 441, 276, 320, 214, 698 , 280, 112 }	
size()	[unchanged]	7
delMax()	{ 441 , 276, 320, 214, 280, 112 }	698
size()	[unchanged]	6
delMax()	{ 276, 320 , 214, 280, 112 }	441
delMax()	{ 276, 214, 280 , 112 }	320
delMax()	{ 276 , 214, 112 }	280
delMax()	{ 214 , 112 }	276
delMax()	{ 112 }	214
size()	[unchanged]	1
delMax()	{ }	112
size()	[unchanged]	0

10.1.5 接口定义

如代码10.1所示，这里以模板类PQ的形式给出以上优先级队列的操作接口定义。

```
1 template <typename T> struct PQ { //优先级队列PQ模板类
2     virtual void insert(T) = 0; //按照比较器确定的优先级次序插入词条
3     virtual T getMax() = 0; //取出优先级最高的词条
4     virtual T delMax() = 0; //删除优先级最高的词条
5 };
```

代码10.1 优先级队列标准接口

因为这一组基本的ADT接口可能有不同的实现方式，故这里均以虚函数形式统一描述这些接口，以便在不同的派生类中具体实现。

10.1.6 应用实例：Huffman编码树

回到5.4节Huffman编码的应用实例。实际上，基于以上优先级队列的标准接口，即可实现统一的Huffman编码算法——无论优先级队列的具体实现方式如何。

■ 数据结构

为利用统一的优先级队列接口实现Huffman编码并对不同方法进行对比，不妨继续沿用代码5.29至代码5.33所定义的Huffman超字符、Huffman树、Huffman森林、Huffman编码表、Huffman二进制编码串等数据结构。

■ 比较器

若将Huffman森林视作优先级队列，则其中每一棵树（每一个超字符）即是一个词条。为保证词条之间可以相互比较，可如代码5.29（145页）所示重载对应的操作符。进一步地，因超字符的优先级可度量为其对应权重的负值，故不妨将大小关系颠倒过来，令小权重超字符的优先级更高，以便于操作接口的统一。

这一技巧也可运用于其它场合。仍以10.1.4节的选择排序为例，在将大小的定义颠倒之后，无需修改其它代码，即可实现反方向的排序。

■ 编码算法

经上述准备，代码10.2即可基于统一优先级队列接口给出通用的Huffman编码算法。

```

1  /*****
2  * Huffman树构造算法：对传入的Huffman森林forest逐步合并，直到成为一棵树
3  *****/
4  * forest基于优先级队列实现，此算法适用于符合PQ接口的任何实现方式
5  * 为Huffman_PQ_List、Huffman_PQ_ComplHeap和Huffman_PQ_LeftHeap共用
6  * 编译前对应工程只需设置相应标志：DSA_PQ_List、DSA_PQ_ComplHeap或DSA_PQ_LeftHeap
7  *****/
8  HuffTree* generateTree(HuffForest* forest) {
9      while (1 < forest->size()) {
10         HuffTree* s1 = forest->delMax(); HuffTree* s2 = forest->delMax();
11         HuffTree* s = new HuffTree();
12         s->insertAsRoot(HuffChar('^', s1->root()->data.weight + s2->root()->data.weight));
13         s->attachAsLC(s->root(), s1); s->attachAsRC(s->root(), s2);
14         forest->insert(s); //将合并后的Huffman树插回Huffman森林
15     }
16     HuffTree* tree = forest->delMax(); //至此，森林中的最后一棵树
17     return tree; //即全局Huffman编码树
18 }

```

代码10.2 利用统一的优先级队列接口，实现通用的Huffman编码

■ 效率分析

相对于如代码5.36（147页）所示的版本，这里只不过将minHChar()替换为PQ::delMax()标准接口。正如我们很快将要看到的，优先级队列的所有ADT操作均可在 $O(\log n)$ 时间内完成，故generateTree()算法也相应地可在 $O(n \log n)$ 时间内构造出Huffman编码树——较之原版本，改进显著。同理，通过引入优先级队列，将如代码3.20（81页）所示的selectMax()替换为PQ::delMax()标准接口，也可自然地将选择排序的性能由 $O(n^2)$ 改进至 $O(n \log n)$ 。

自然地，这一结论可以推广至任一需要反复选取优先级最高元素的应用问题，并可直接改进相关算法的时间效率。那么，作为基础性数据结构的优先级队列，是否的确可以保证getMax()、delMax()和insert()等接口效率均为 $O(\log n)$ ？具体地，又应如何实现？

实际上，借助无序列表、有序列表、无序向量或有序向量，都难以同时兼顾insert()和delMax()操作的高效率（习题[10-1]）。因此，必须另辟蹊径，寻找更为高效的实现方法。

§ 10.2 堆

基于列表或向量等结构的实现方式，之所以无法同时保证`insert()`和`delMax()`操作的高效率，原因在于其对优先级的理解过于机械，以致始终都保存了全体词条之间的全序关系。实际上，尽管优先级队列的确隐含了“所有词条可相互比较”这一条件，但从操作接口层面来看，并不需要真正地维护全序关系。比如执行`delMax()`操作时，只要能够确定全局优先级最高的词条即可；至于次高者、第三高者等其余词条，目前暂时不必关心。

有限偏序集的极值必然存在，故此时借助堆（heap）结构维护一个偏序关系即足矣。堆有多种实现形式，以下首先介绍其中最基本的一种形式——完全二叉堆（complete binary heap）。

10.2.1 完全二叉堆

■ 结构性与堆序性

如图10.1实例所示，完全二叉堆应满足两个条件。

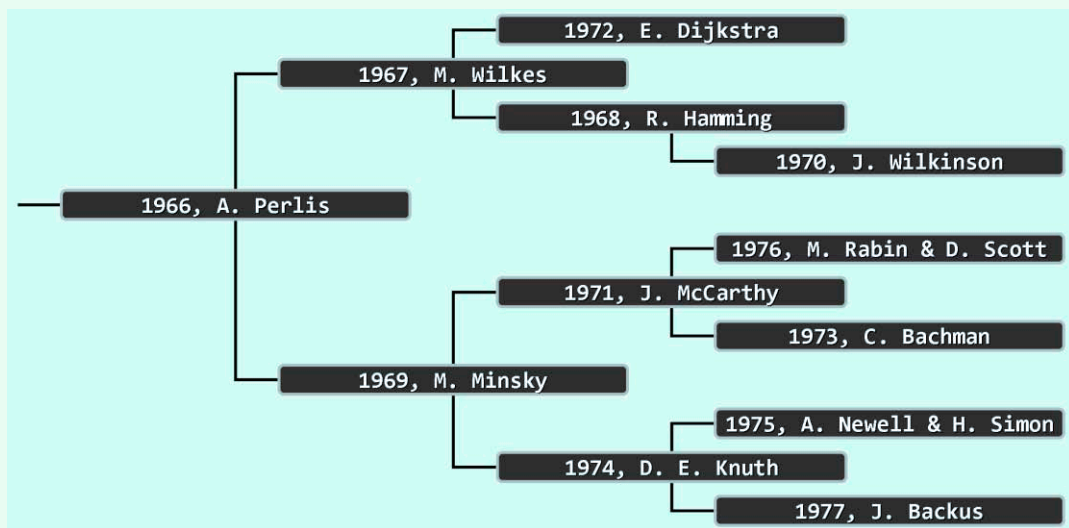


图10.1 以获奖先后为优先级，由前12届图灵奖得主构成的完全二叉堆

首先，其逻辑结构须等同于完全二叉树，此即所谓的“结构性”。如此，堆节点将与词条一一对应，故此后凡不致引起误解时，我们将不再严格区分“堆节点”与“词条”。其次，就优先级而言，堆顶以外的每个节点都不高（大）于其父节点，此即所谓的“堆序性”。

■ 大顶堆与小顶堆

由堆序性不难看出，堆中优先级最高的词条必然始终处于堆顶位置。因此，堆结构的`getMax()`操作总是可以在 $O(1)$ 时间内完成。

堆序性也可对称地约定为“堆顶以外的每个节点都不低（小）于其父节点”，此时同理，优先级最低的词条，必然始终处于堆顶位置。为以示区别，通常称前（后）者为大（小）顶堆。

小顶堆和大顶堆是相对的，而且可以相互转换。实际上，我们不久之前刚刚见过这样的实例——在代码5.29中重载Huffman超字符的比较操作符时，通过对超字符权重取负，颠倒优先级关系，使之与算法的实际语义及需求相吻合。